

# CS Master Notes — Module 2: Linked Lists

RV University | 2nd Semester Computer Science | CIE 1 Preparation Reference: [W3Schools DSA — Linked Lists](#) | Subject Matter Expert Edition

---

## 1. Introduction to Linked Lists

### 1.1 Definition & Motivation

A **linked list** is a linear, dynamic data structure in which elements (called **nodes**) are stored at non-contiguous memory locations. Each node contains:

1. **Data field** — the value stored
2. **Pointer field(s)** — address(es) of adjacent node(s)

Unlike arrays, a linked list does **not** require contiguous memory. Nodes are linked via pointers, forming a chain.

#### Motivation — Why linked lists over arrays?

- Arrays require a **fixed size** declared at compile time (static) or a costly realloc (dynamic)
- Inserting/deleting in the middle of an array requires  **$O(n)$  element shifts**
- Linked lists support  **$O(1)$  insertion/deletion** at a known position (given the predecessor pointer)
- Linked lists grow and shrink at **runtime** without reallocation of the entire structure

## 1.2 Array vs. Linked List — Comparative Analysis

| Dimension              | Array                                     | Linked List                                       |
|------------------------|-------------------------------------------|---------------------------------------------------|
| Memory layout          | Contiguous                                | Non-contiguous (scattered on heap)                |
| Size                   | Fixed (static) or costly resize (dynamic) | Dynamic — grows/shrinks at runtime                |
| Access by index        | $O(1)$ — direct address computation       | $O(n)$ — sequential pointer traversal             |
| Search (unsorted)      | $O(n)$                                    | $O(n)$                                            |
| Search (sorted)        | $O(\log n)$ binary search                 | $O(n)$ — no random access                         |
| Insert at beginning    | $O(n)$ — shift all elements               | $O(1)$ — pointer update only                      |
| Insert at end          | $O(1)$ amortized (dynamic array)          | $O(n)$ SLL (no tail ptr); $O(1)$ with tail ptr    |
| Insert at middle       | $O(n)$ — shift                            | $O(n)$ search + $O(1)$ pointer update             |
| Delete at beginning    | $O(n)$ — shift                            | $O(1)$                                            |
| Delete at middle       | $O(n)$                                    | $O(n)$ search + $O(1)$ pointer update             |
| Memory per element     | <code>sizeof(T)</code>                    | <code>sizeof(T) + sizeof(pointer) × 1 or 2</code> |
| Cache performance      | Excellent — spatial locality              | Poor — pointer chasing, cache misses              |
| Supports binary search | Yes                                       | No (no $O(1)$ index access)                       |
| Overflow risk          | Stack overflow / realloc failure          | Heap exhaustion (each node is malloc'd)           |

### 1.3 Node Structure

Every linked list node is a **self-referential structure** — a struct containing a pointer to the same struct type.

**Singly linked node:**

**Doubly linked node:**

**Dynamic node creation (malloc):**

```
struct Node* createNode(int val) {
    struct Node* newNode = (struct Node*) malloc(sizeof(struct
Node));
    if (newNode == NULL) {
        fprintf(stderr, "Memory allocation failed\n");
        exit(EXIT_FAILURE);
    }
    newNode->data = val;
    newNode->next = NULL;
    return newNode;
}
```

---

## 2. Singly Linked List (SLL)

### 2.1 Structure & Memory Layout

A **Singly Linked List** is a linked list where each node contains exactly **one pointer** (next) pointing to the successor node. The last node's next pointer is NULL, marking the end of the list.

HEAD

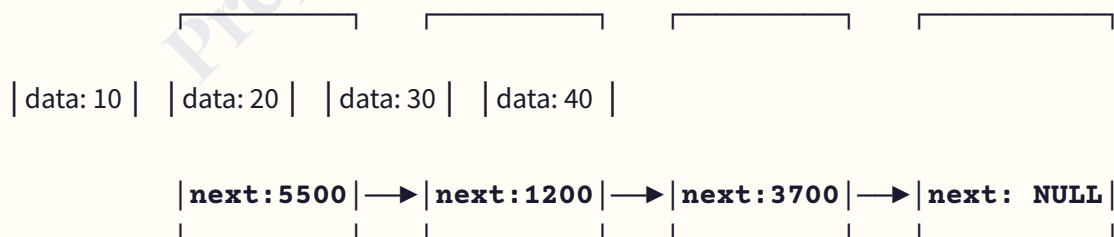


[10 | •] → [20 | •] → [30 | •] → [40 | NULL]

- HEAD — pointer variable holding the address of the first node
- Each • represents the next pointer field
- Last node: next = NULL

**Memory representation (non-contiguous):**

Address: 2000      5500      1200      3700



HEAD = 2000

Visual: Singly Linked List Diagram

**C Structure Definition:**

```
#include <stdio.h>
#include <stdlib.h>
```

```
// Convenience typedef
typedef struct Node Node;
```

---

## 2.2 Create

**Operation:** Allocate a new node and initialize it; this forms the foundation of all insert operations.

```
// Creates a single node with given value; next = NULL
Node* createNode(int val) {
    Node* newNode = (Node*) malloc(sizeof(Node));
    if (!newNode) { perror("malloc"); exit(EXIT_FAILURE); }
    newNode->data = val;
    newNode->next = NULL;
    return newNode;
}
```

Create a list from an array of values:

```
// Returns HEAD of the new list
Node* createList(int arr[], int n) {
    if (n == 0) return NULL;

    Node* head = createNode(arr[0]);
```

```

Node* current = head;

for (int i = 1; i < n; i++) {
    current->next = createNode(arr[i]);
    current = current->next;
}
return head;
}

```

```

// Example:
// int vals[] = {10, 20, 30, 40};
// Node* head = createList(vals, 4);
// Result: 10 → 20 → 30 → 40 → NULL

```

Time Complexity

Space Complexity

$O(n)$

$O(n)$  —  $n$  nodes allocated on heap

## 2.3 Traverse

Operation: Visit every node exactly once, sequentially following next pointers from head to the node where `next == NULL`.

```

void traverse(Node* head) {
    Node* current = head;
    printf("List: ");
    while (current != NULL) {
        printf("%d", current->data);
        if (current->next != NULL) printf(" → ");
        current = current->next;    // Advance pointer
    }
}

```

```
    printf(" → NULL\n");  
}
```

```
// Example:  
// head → [10] → [20] → [30] → [40] → NULL  
// Output: List: 10 → 20 → 30 → 40 → NULL
```

### Recursive traversal:

```
void traverseRecursive(Node* head) {  
    if (head == NULL) { printf("NULL\n"); return; }  
    printf("%d → ", head->data);  
    traverseRecursive(head->next);  
}
```

| Time Complexity | Space Complexity            |
|-----------------|-----------------------------|
| O(n) iterative  | O(1) iterative              |
| O(n) recursive  | O(n) recursive (call stack) |

## 2.4 Search

**Operation:** Find the first node whose data field equals key. Returns the node pointer (or NULL if not found).

```
// Returns pointer to first node with data == key; NULL if not  
found  
Node* search(Node* head, int key) {  
    Node* current = head;
```

```

while (current != NULL) {
    if (current->data == key) return current; // Found
    current = current->next;
}
return NULL; // Not found
}

```

```

// Search returning index (0-based):
int searchIndex(Node* head, int key) {
    int idx = 0;
    Node* current = head;
    while (current != NULL) {
        if (current->data == key) return idx;
        current = current->next;
        idx++;
    }
}

```

```

}
return -1; // Not found
}

```

```

// Example:
// List: 10 → 20 → 30 → 40 → NULL, key = 30
// idx=0: 10≠30, idx=1: 20≠30, idx=2: 30=30 → return 2

```

Case

Time Complexity

|                                  |                 |
|----------------------------------|-----------------|
| Best (key at head)               | $O(1)$          |
| Average                          | $O(n/2) = O(n)$ |
| Worst (key at tail or not found) | $O(n)$          |

---

## 2.5 Insert

### 2.5.1 Insert at Beginning (Prepend)

#### Algorithm:

1. Create new node
2. Set `newNode->next = head`
3. Update `head = newNode`

```
Node* insertAtBeginning(Node* head, int val) {
    Node* newNode = createNode(val);
    newNode->next = head;    // New node points to old head
    return newNode;        // New node IS the new head
}
```

```
// Example:
// Before: head → [20] → [30] → NULL
// insert 10 at beginning
// After:  head → [10] → [20] → [30] → NULL
```

### 2.5.2 Insert at End (Append)

#### Algorithm:

1. Create new node
2. If list is empty, new node becomes head
3. Else traverse to the last node (where next == NULL)
4. Set last->next = newNode

```
Node* insertAtEnd(Node* head, int val) {  
    Node* newNode = createNode(val);  
    if (head == NULL) return newNode; // Empty list
```

```
    Node* current = head;  
    while (current->next != NULL) // Traverse to last node  
        current = current->next;
```

```
    current->next = newNode; // Append  
    return head;  
}
```

```
// Example:  
// Before: head → [10] → [20] → NULL  
// insert 30 at end  
// After: head → [10] → [20] → [30] → NULL
```

### 2.5.3 Insert at Given Position (0-based index)

**Algorithm:**

1. If pos == 0, use insertAtBeginning
2. Else traverse to node at pos-1 (the predecessor)
3. Set newNode->next = predecessor->next
4. Set predecessor->next = newNode

```
Node* insertAtPosition(Node* head, int pos, int val) {
    if (pos == 0) return insertAtBeginning(head, val);

    Node* newNode = createNode(val);
    Node* current = head;

    for (int i = 0; i < pos - 1; i++) {
        if (current == NULL) {
            printf("Position %d out of bounds\n", pos);
            free(newNode);
            return head;
        }
        current = current->next;
    }
}
```

```
    newNode->next = current->next; // Step 1: link new node
forward
    current->next = newNode;       // Step 2: link predecessor
to new node
    return head;
}
```

```
// Example:
// Before: head → [10] → [20] → [40] → NULL
// insertAtPosition(head, 2, 30) - insert 30 at index 2
// Traverse: current = node[10] (i=0), current = node[20] (i=1) →
```

stop at pos-1=1

```
// newNode(30)->next = node[20]->next = node[40]
// node[20]->next = newNode(30)
// After: head → [10] → [20] → [30] → [40] → NULL
```

**Critical pointer order:** Always set `newNode->next` BEFORE `predecessor->next = newNode`. If reversed, the forward chain is lost.

| Insert Position      | Time Complexity |
|----------------------|-----------------|
| At beginning         | O(1)            |
| At end (no tail ptr) | O(n)            |
| At position i        | O(i)            |

## 2.6 Delete

### 2.6.1 Delete at Beginning

```
Node* deleteAtBeginning(Node* head) {
    if (head == NULL) { printf("List is empty\n"); return NULL; }
    Node* temp = head;
    head = head->next;    // Move head to second node
    free(temp);          // Deallocate old head
    return head;
}
```

```
// Example:
// Before: head → [10] → [20] → [30] → NULL

// After:  head → [20] → [30] → NULL    (node 10 freed)
```

### 2.6.2 Delete at End

```
Node* deleteAtEnd(Node* head) {
    if (head == NULL) { printf("List is empty\n"); return NULL; }
    if (head->next == NULL) { free(head); return NULL; } //
Single node
```

```
    Node* current = head;
    while (current->next->next != NULL) // Stop at
second-to-last
        current = current->next;

    free(current->next); // Free last node
    current->next = NULL; // Update second-to-last to be new
last
    return head;
}
```

```
// Example:
// Before: head → [10] → [20] → [30] → NULL
```

```
// Traverse: current stops at node[20] (current->next->next ==
NULL)
// free(node[30]), node[20]->next = NULL
// After: head → [10] → [20] → NULL
```

### 2.6.3 Delete by Value

```
Node* deleteByValue(Node* head, int key) {
    if (head == NULL) return NULL;
```

// Case 1: Head node contains key

```
    if (head->data == key) {
        Node* temp = head;
        head = head->next;
        free(temp);
        return head;
    }
```

// Case 2: Key is in the rest of the list

```
    Node* current = head;
    while (current->next != NULL) {
        if (current->next->data == key) {
            Node* temp = current->next;           // Node to
delete
            current->next = current->next->next; // Bypass
deleted node
            free(temp);
```

```

        return head;
    }
    current = current->next;
}

printf("Value %d not found\n", key);
return head;
}

```

**// Example:**  
**// Before: head → [10] → [20] → [30] → [40] → NULL, key=30**

```

// current=node[10]: current->next=node[20], data=20≠30
// current=node[20]: current->next=node[30], data=30=30 → delete
// node[20]->next = node[30]->next = node[40]
// free(node[30])
// After: head → [10] → [20] → [40] → NULL

```

#### 2.6.4 Delete at Position

```

Node* deleteAtPosition(Node* head, int pos) {
    if (head == NULL) return NULL;
    if (pos == 0) return deleteAtBeginning(head);

```

```

    Node* current = head;
    for (int i = 0; i < pos - 1; i++) {
        if (current->next == NULL) {

```

```

        printf("Position %d out of bounds\n", pos);
        return head;
    }
    current = current->next;
}

```

```

Node* temp = current->next;
if (temp == NULL) { printf("Position out of bounds\n");
return head; }
current->next = temp->next;
free(temp);
return head;
}

```

| Delete Position | Time Complexity |
|-----------------|-----------------|
| At beginning    | O(1)            |
| At end          | O(n)            |
| At position i   | O(i)            |
| By value        | O(n)            |

## 2.7 Update

**Operation:** Find the node at a given position (or with a given value) and modify its data field.

```

// Update by position (0-based)
int updateAtPosition(Node* head, int pos, int newVal) {
    Node* current = head;
    for (int i = 0; i < pos; i++) {
        if (current == NULL) { printf("Position out of
bounds\n"); return -1; }
        current = current->next;
    }
    if (current == NULL) { printf("Position out of bounds\n");
return -1; }
    current->data = newVal;
    return 0;
}

// Update by value (first occurrence)
int updateByValue(Node* head, int oldVal, int newVal) {
    Node* node = search(head, oldVal);
    if (node == NULL) { printf("%d not found\n", oldVal); return
-1; }
    node->data = newVal;
    return 0;}

```

```

// Example:
// List: 10 → 20 → 30 → NULL
// updateAtPosition(head, 1, 99) → 10 → 99 → 30 → NULL

```

| Time Complexity                    | Space Complexity |
|------------------------------------|------------------|
| $O(n)$ — must traverse to position | $O(1)$           |

## 2.8 Complete SLL Program

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node* next;
} Node;
```

```
Node* createNode(int val) {
    Node* n = (Node*) malloc(sizeof(Node));
    if (!n) { perror("malloc"); exit(1); }
    n->data = val; n->next = NULL;
    return n;
}

void traverse(Node* head) {
    printf("List: ");
    for (Node* c = head; c; c = c->next)
        printf("%d%s", c->data, c->next ? " → " : " → NULL\n");
    if (!head) printf("NULL\n");
}

Node* insertAtBeginning(Node* head, int val) {
    Node* n = createNode(val);
    n->next = head;
    return n;
}

Node* insertAtEnd(Node* head, int val) {
    Node* n = createNode(val);
    if (!head) return n;
```

```

    Node* c = head;
    while (c->next) c = c->next;
    c->next = n;
    return head;
}

Node* insertAtPosition(Node* head, int pos, int val) {
    if (pos == 0) return insertAtBeginning(head, val);
    Node* n = createNode(val);
    Node* c = head;
    for (int i = 0; i < pos - 1 && c; i++) c = c->next;
    if (!c) { printf("Out of bounds\n"); free(n); return head; }
    n->next = c->next; c->next = n;
    return head;
}

Node* deleteByValue(Node* head, int key) {
    if (!head) return NULL;
    if (head->data == key) { Node* t = head; head = head->next;
    free(t); return head; }
    Node* c = head;
    while (c->next && c->next->data != key) c = c->next;
    if (c->next) { Node* t = c->next; c->next = t->next; free(t);
    }
    else printf("%d not found\n", key);
    return head;
}

Node* search(Node* head, int key) {
    for (Node* c = head; c; c = c->next)
        if (c->data == key) return c;
    return NULL;
}

```

```
int main() {  
    Node* head = NULL;
```

// Create / Insert

```
    head = insertAtEnd(head, 10);  
    head = insertAtEnd(head, 20);  
    head = insertAtEnd(head, 40);  
    head = insertAtEnd(head, 50);  
    printf("After creation:\n"); traverse(head);
```

// Insert at position

```
    head = insertAtPosition(head, 2, 30);  
    printf("After insert 30 at pos 2:\n"); traverse(head);
```

// Search

```
    Node* found = search(head, 30);  
    printf("Search 30: %s\n", found ? "Found" : "Not Found");
```

// Delete

```
    head = deleteByValue(head, 30);  
    printf("After delete 30:\n"); traverse(head);
```

// Update

```
    head->next->data = 99; // Update second node
    printf("After update index 1 to 99:\n"); traverse(head);

    return 0;
}
```

/\* Expected Output:

After creation:

List: 10 → 20 → 40 → 50 → NULL

After insert 30 at pos 2:

List: 10 → 20 → 30 → 40 → 50 → NULL

Search 30: Found

After delete 30:

List: 10 → 20 → 40 → 50 → NULL

After update index 1 to 99:

List: 10 → 99 → 40 → 50 → NULL

\*/

---

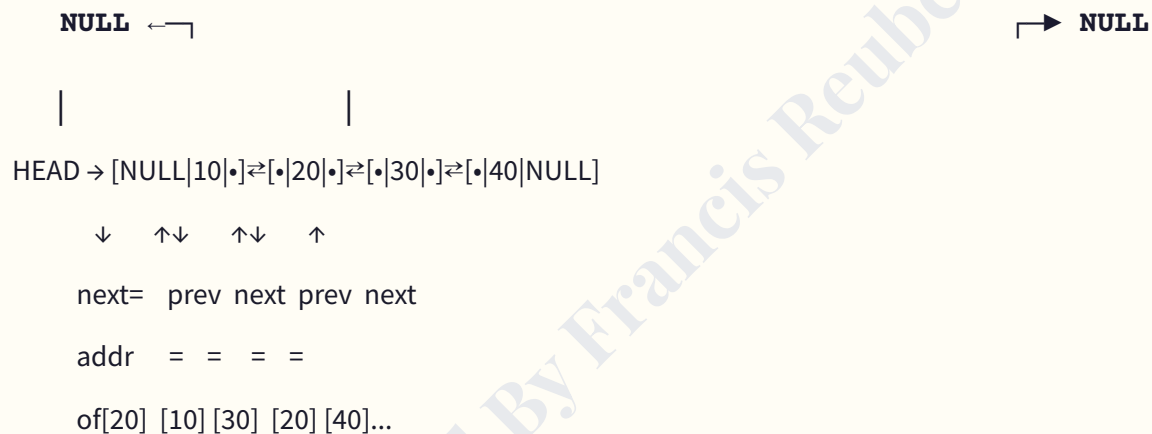
### 3. Doubly Linked List (DLL)

## 3.1 Structure & Memory Layout

A **Doubly Linked List** is a linked list where each node contains **two pointers**:

- prev — pointer to the predecessor node (NULL for the head node)
- next — pointer to the successor node (NULL for the tail node)

This enables **bidirectional traversal** — forward and backward.



Visual: Doubly Linked List Diagram

### C Structure:

```
typedef struct DNode {
    int data;
    struct DNode* prev;
    struct DNode* next;
} DNode;
```

```

DNode* createDNode(int val) {
    DNode* n = (DNode*) malloc(sizeof(DNode));
    if (!n) { perror("malloc"); exit(1); }
    n->data = val;
    n->prev = NULL;
    n->next = NULL;
    return n;
}

```

**Key advantage over SLL:** Deletion of a given node is **O(1)** if you already have the pointer to it (no need to find predecessor — prev is stored in the node itself).

---

### 3.2 Create

```

// Create DLL from array; returns HEAD
DNode* createDLL(int arr[], int n) {
    if (n == 0) return NULL;
    DNode* head = createDNode(arr[0]);
    DNode* current = head;

    for (int i = 1; i < n; i++) {
        DNode* newNode = createDNode(arr[i]);
        current->next = newNode;    // Forward link
        newNode->prev = current;    // Backward link
        current = newNode;
    }
    return head;
}

```

```
// Example:  
// int vals[] = {10, 20, 30};  
// DNode* head = createDLL(vals, 3);  
// NULL ← [10] ⇌ [20] ⇌ [30] → NULL
```

---

### 3.3 Traverse (Forward & Backward)

// Forward traversal: HEAD to TAIL

```
void traverseForward(DNode* head) {  
    printf("Forward:  NULL ← ");  
    DNode* current = head;  
    DNode* last = NULL;  
    while (current != NULL) {  
        printf("[%d]", current->data);  
        if (current->next) printf(" ⇌ ");  
        last = current;  
    }
```

```
        current = current->next;  
    }  
    printf(" → NULL\n");  
}
```

```
// Backward traversal: TAIL to HEAD  
// Requires pointer to tail (or traverse forward first to find
```

```

tail)
void traverseBackward(DNode* tail) {
    printf("Backward: NULL ← ");
    DNode* current = tail;
    while (current != NULL) {
        printf("[%d]", current->data);
        if (current->prev) printf(" ⇐ ");
        current = current->prev;
    }
    printf(" → NULL\n");
}

```

```

// Helper: get tail
DNode* getTail(DNode* head) {
    if (!head) return NULL;
    DNode* c = head;
    while (c->next) c = c->next;
    return c;
}

```

---

### 3.4 Search

```

DNode* searchDLL(DNode* head, int key) {
    DNode* current = head;
    while (current != NULL) {
        if (current->data == key) return current;
        current = current->next;
    }
}

```

```
    return NULL;
}
```

Time Complexity

Space Complexity

O(n)

O(1)

### 3.5 Insert

#### 3.5.1 Insert at Beginning

```
DNode* insertDLLAtBeginning(DNode* head, int val) {
    DNode* newNode = createDNode(val);
    if (head != NULL) {
        newNode->next = head;    // new → old head
        head->prev = newNode;    // old head ← new
    }
    return newNode;              // new node is new head
}
```

```
// Example:
// Before: NULL ← [20] ⇌ [30] → NULL
// Insert 10 at beginning
// After:  NULL ← [10] ⇌ [20] ⇌ [30] → NULL
```

#### 3.5.2 Insert at End

```
DNode* insertDLLAtEnd(DNode* head, int val) {
    DNode* newNode = createDNode(val);
    if (!head) return newNode;
```

```
DNode* current = head;
while (current->next) current = current->next; // Reach tail
```

```
current->next = newNode; // tail → new
newNode->prev = current; // new ← tail
return head;
}
```

### 3.5.3 Insert After a Given Node

```
// Inserts newVal AFTER node pointed to by 'target'
void insertAfterDLL(DNode* target, int val) {
    if (!target) return;
    DNode* newNode = createDNode(val);
```

```
newNode->next = target->next; // new → target's successor
newNode->prev = target;       // new ← target

if (target->next)
    target->next->prev = newNode; // successor ← new
```

```
target->next = newNode; // target → new
}
```

```
// Example:
// List: NULL ← [10] ⇌ [20] ⇌ [40] → NULL

// insertAfterDLL(node[20], 30)
// After: NULL ← [10] ⇌ [20] ⇌ [30] ⇌ [40] → NULL
```

4-step pointer sequence for insert-after (memorize this):

1. newNode->next = target->next
2. newNode->prev = target
3. if (target->next) target->next->prev = newNode
4. target->next = newNode

---

## 3.6 Delete

### 3.6.1 Delete a Given Node (O(1) – key DLL advantage)

// Deletes node pointed to by 'del' from DLL; returns new head

```
DNode* deleteDLLNode(DNode* head, DNode* del) {
    if (!head || !del) return head;

    if (head == del) head = del->next;    // Deleting head

    if (del->next) del->next->prev = del->prev; // successor's
    prev bypasses del
    if (del->prev) del->prev->next = del->next; // predecessor's
    next bypasses del

    free(del);
```

```

    return head;
}

```

```

// Example:
// List: NULL ← [10] ⇌ [20] ⇌ [30] → NULL, del = node[20]

```

```

// node[30] -> prev = node[20] -> prev = node[10]
// node[10] -> next = node[20] -> next = node[30]
// free(node[20])
// After: NULL ← [10] ⇌ [30] → NULL

```

### 3.6.2 Delete by Value

```

DNode* deleteByValueDLL(DNode* head, int key) {
    DNode* node = searchDLL(head, key);
    if (!node) { printf("%d not found\n", key); return head; }
    return deleteDLLNode(head, node);
}

```

| Delete Type               | SLL Time                | DLL Time                | Reason                |
|---------------------------|-------------------------|-------------------------|-----------------------|
| Delete node given pointer | O(n) — find predecessor | O(1) — use prev pointer | DLL stores prev       |
| Delete by value           | O(n)                    | O(n)                    | Search still required |
| Delete at beginning       | O(1)                    | O(1)                    | Same                  |
| Delete at end             | O(n)                    | O(n) — no tail ptr      | Traverse to tail      |

### 3.7 Update

```
int updateDLL(DNode* head, int pos, int newVal) {
    DNode* current = head;
    for (int i = 0; i < pos; i++) {
        if (!current) { printf("Out of bounds\n"); return -1; }
        current = current->next;
    }
    if (!current) { printf("Out of bounds\n"); return -1; }
    current->data = newVal;
    return 0;
}
```

---

### 3.8 Complete DLL Program

```
#include <stdio.h>
#include <stdlib.h>

typedef struct DNode {
    int data;
    struct DNode* prev;
    struct DNode* next;
} DNode;
```

```
DNode* createDNode(int val) {
    DNode* n = (DNode*) malloc(sizeof(DNode));
    if (!n) { perror("malloc"); exit(1); }
    n->data = val; n->prev = n->next = NULL;
    return n;
}
```

```
}
```

```
void traverseForward(DNode* head) {  
    printf("Forward: ");  
    for (DNode* c = head; c; c = c->next)  
        printf("[%d]%s", c->data, c->next ? " > " : "\n");  
    if (!head) printf("Empty\n");  
}
```

```
DNode* insertAtBeginning(DNode* head, int val) {  
    DNode* n = createDNode(val);  
    if (head) { n->next = head; head->prev = n; }  
    return n;  
}
```

```
DNode* insertAtEnd(DNode* head, int val) {  
    DNode* n = createDNode(val);  
    if (!head) return n;  
    DNode* c = head;  
    while (c->next) c = c->next;  
    c->next = n; n->prev = c;  
    return head;  
}
```

```
DNode* deleteByValue(DNode* head, int key) {  
    DNode* c = head;  
    while (c && c->data != key) c = c->next;  
    if (!c) { printf("%d not found\n", key); return head; }  
    if (head == c) head = c->next;  
    if (c->next) c->next->prev = c->prev;  
    if (c->prev) c->prev->next = c->next;  
    free(c);  
    return head;  
}
```

```

}

int main() {
    DNode* head = NULL;
    head = insertAtEnd(head, 10);
    head = insertAtEnd(head, 20);
    head = insertAtEnd(head, 30);
    head = insertAtEnd(head, 40);
    printf("Created: "); traverseForward(head);

    head = insertAtBeginning(head, 5);
    printf("Insert 5 at beginning: "); traverseForward(head);

    head = deleteByValue(head, 20);
    printf("Delete 20: "); traverseForward(head);

    return 0;
}

```

/\* Expected Output:

Created: [10] ⇌ [20] ⇌ [30] ⇌ [40]

Insert 5 at beginning: [5] ⇌ [10] ⇌ [20] ⇌ [30] ⇌ [40]

Delete 20: [5] ⇌ [10] ⇌ [30] ⇌ [40]

\*/

---

#### 4. Circular Singly Linked List (CSLL)

##### 4.1 Structure & Memory Layout

A **Circular Singly Linked List** is a singly linked list where the next pointer of the **last node** points back to the **first node** (HEAD), forming a closed loop. There is **no NULL** in the next field of any node.

HEAD

|



(last node's next = HEAD = node[10])

Visual: Circular Singly Linked List

##### Key properties:

- No NULL pointer in the list (except in empty list: head = NULL)
- Traversal must use a **stopping condition** other than current == NULL
- Commonly maintained with a **tail pointer** for O(1) insertion at both ends

**Stopping condition for traversal: current->next == head (one full loop completed)**

## 4.2 Create

```
typedef struct CNode {
    int data;
    struct CNode* next;
} CNode;
```

```
CNode* createCNode(int val) {
    CNode* n = (CNode*) malloc(sizeof(CNode));
    if (!n) { perror("malloc"); exit(1); }
    n->data = val;
    n->next = n;    // Points to itself - forms a circle of 1
    return n;
}
```

```
// Create CSLL from array; returns HEAD
CNode* createCSLL(int arr[], int n) {
    if (n == 0) return NULL;
    CNode* head = createCNode(arr[0]);
    CNode* tail = head;

    for (int i = 1; i < n; i++) {
        CNode* newNode = (CNode*) malloc(sizeof(CNode));
        newNode->data = arr[i];
        newNode->next = head;    // New node loops back to head
        tail->next = newNode;    }
    return head; } // Old tail points to new node
```

```
tail = newNode;    // Update tail
```

---

### 4.3 Traverse

Critical: Must stop when `current->next == head`, NOT when `current == NULL`.

```
void traverseCSLL(CNode* head) {
    if (!head) { printf("Empty list\n"); return; }
    printf("Circular List: ");
    CNode* current = head;
    do {
        printf("%d → ", current->data);
        current = current->next;
    } while (current != head);
    printf("(back to %d)\n", head->data);
}
```

// Example:

// CSLL: 10 → 20 → 30 → (back to 10)

// Output: Circular List: 10 → 20 → 30 → (back to 10)

---

### 4.4 Search

```
CNode* searchCSLL(CNode* head, int key) {
    if (!head) return NULL;
    CNode* current = head;
    do {
        if (current->data == key) return current;
        current = current->next;
    } while (current != head);
    return NULL;
}
```

## 4.5 Insert

### 4.5.1 Insert at Beginning

```
CNode* insertCSLLAtBeginning(CNode* head, int val) {
    CNode* newNode = createCNode(val);
    if (!head) return newNode; // newNode->next = itself already

    // Find tail (node whose next == head)
    CNode* tail = head;
    while (tail->next != head) tail = tail->next;
```

```
    newNode->next = head; // new → old head
    tail->next = newNode; // tail → new (maintains circularity)
    return newNode;      // new is the new head
}
```

### 4.5.2 Insert at End

**Efficient approach using tail pointer** (traverse to find tail, insert after it):

```
CNode* insertCSLLAtEnd(CNode* head, int val) {
    CNode* newNode = (CNode*) malloc(sizeof(CNode));
    newNode->data = val;

    if (!head) {
        newNode->next = newNode; // Circular self-loop
        return newNode;
    }
```

```
CNode* tail = head;
while (tail->next != head) tail = tail->next;
```

```
tail->next = newNode;    // old tail → new
newNode->next = head;    // new → head (maintains
circularity)
return head;
}
```

```
// Example:
// Before: 10 → 20 → (10), insert 30 at end
```

```
// tail = node[20], tail->next=node[10] (stop condition)
// node[20]->next = node[30]
// node[30]->next = head = node[10]
// After: 10 → 20 → 30 → (10)
```

#### 4.5.3 Insert at Position

```
CNode* insertCSLLAtPosition(CNode* head, int pos, int val) {
    if (pos == 0) return insertCSLLAtBeginning(head, val);

    CNode* newNode = (CNode*) malloc(sizeof(CNode));
    newNode->data = val;
```

```

CNode* current = head;
for (int i = 0; i < pos - 1; i++) {
    current = current->next;
    if (current == head) {
        printf("Position out of bounds\n");
        free(newNode); return head;
    }
}

```

```

newNode->next = current->next;
current->next = newNode;
return head;
}

```

## 4.6 Delete

### 4.6.1 Delete at Beginning

```

CNode* deleteCSLLAtBeginning(CNode* head) {
    if (!head) return NULL;
    if (head->next == head) { free(head); return NULL; } //
Single node

```

```

CNode* tail = head;
while (tail->next != head) tail = tail->next;

```

```

CNode* temp = head;
head = head->next;    // Move head forward
tail->next = head;    // Tail now points to new head
free(temp);
return head;
}

```

#### 4.6.2 Delete by Value

```

CNode* deleteCSLLByValue(CNode* head, int key) {
    if (!head) return NULL;

```

// Case: head node contains key

```

    if (head->data == key) return deleteCSLLAtBeginning(head);

```

```

CNode* current = head;
do {
    if (current->next->data == key) {
        CNode* temp = current->next;
        current->next = temp->next;
        free(temp);
        return head;
    }
    current = current->next;
} while (current->next != head);

printf("%d not found\n", key);
return head;}

```

## 4.7 Update

```
int updateCSLL(CNode* head, int pos, int newVal) {
    if (!head) return -1;
    CNode* current = head;
    for (int i = 0; i < pos; i++) {
        current = current->next;
        if (current == head) { printf("Out of bounds\n"); return
-1; }
    }
    current->data = newVal;
    return 0;}

```

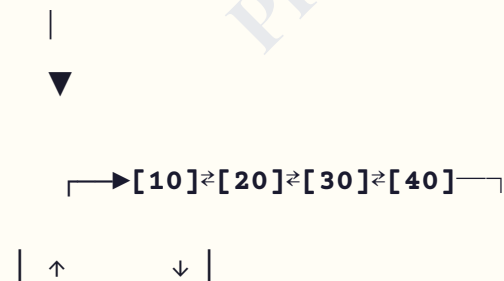
## 5. Circular Doubly Linked List (CDLL)

## 5.1 Structure & Memory Layout

A **Circular Doubly Linked List** combines the properties of both circular and doubly linked lists:

- Each node has both prev and next pointers
- The **head node's prev** points to the **tail node**
- The **tail node's next** points to the **head node**
- No NULL anywhere in a non-empty list

HEAD



```
head->prev = tail    tail->next = head
```

## Visual: Circular Doubly Linked List

### C Structure:

```
typedef struct CDNode {
    int data;
    struct CDNode* prev;
    struct CDNode* next;
} CDNode;
```

### 5.2 Create

```
CDNode* createCDNode(int val) {
    CDNode* n = (CDNode*) malloc(sizeof(CDNode));
    if (!n) { perror("malloc"); exit(1); }
    n->data = val;
    n->prev = n;    // Points to itself
    n->next = n;    // Points to itself
    return n;
}

CDNode* createCDLL(int arr[], int n) {
    if (n == 0) return NULL;
    CDNode* head = createCDNode(arr[0]);

    for (int i = 1; i < n; i++) {
        CDNode* newNode = createCDNode(arr[i]);
        CDNode* tail = head->prev;    // head->prev always points
to tail

```

// Insert newNode between tail and head

```
        tail->next = newNode;
        newNode->prev = tail;
        newNode->next = head;
        head->prev = newNode;
    }
    return head;
}
```

---

### 5.3 Traverse

```
void traverseCDLL(CDNode* head) {
    if (!head) { printf("Empty\n"); return; }
    printf("CDLL: ");
    CDNode* current = head;
    do {
        printf("[%d]", current->data);
        if (current->next != head) printf(" ⇌ ");
        current = current->next;
    } while (current != head);
    printf(" ⇌ (circular)\n");
}
```

---

### 5.4 Insert

#### 5.4.1 Insert at Beginning

```
CDNode* insertCDLLAtBeginning(CDNode* head, int val) {
    CDNode* newNode = createCDNode(val);
```

```
if (!head) return newNode;
```

```
CDNode* tail = head->prev;
```

// Wire newNode between tail and head

```
newNode->next = head;  
newNode->prev = tail;  
tail->next = newNode;  
head->prev = newNode;  
  
return newNode; // New head  
}
```

#### 5.4.2 Insert at End

```
CDNode* insertCDLLAtEnd(CDNode* head, int val) {  
    CDNode* newNode = createCDNode(val);  
    if (!head) return newNode;
```

```
CDNode* tail = head->prev;
```

```
tail->next = newNode;  
newNode->prev = tail;  
newNode->next = head;
```

```
    head->prev = newNode;

    return head;    // Head unchanged
}
```

```
// Example:
// Before: [10] ⇌ [20] ⇌ (circular), insert 30 at end
```

```
// tail = head->prev = node[20]
// node[20]->next = node[30]
// node[30]->prev = node[20]
// node[30]->next = head = node[10]
// node[10]->prev = node[30]
// After: [10] ⇌ [20] ⇌ [30] ⇌ (circular)
```

---

## 5.5 Delete

### 5.5.1 Delete a Node (O(1) – given pointer)

```
CDNode* deleteCDLLNode(CDNode* head, CDNode* del) {
    if (!head || !del) return head;
```

```
// Only one node
```

```
    if (head == del && head->next == head) {
        free(del);
        return NULL;
```

```
}
```

```
CDNode* prevNode = del->prev;  
CDNode* nextNode = del->next;
```

```
prevNode->next = nextNode;  
nextNode->prev = prevNode;  
  
if (head == del) head = nextNode; // Update head if needed  
  
free(del);  
return head;  
}
```

```
// Example:  
// CDLL: [10] ⇌ [20] ⇌ [30] ⇌ (circular), delete node[20]  
// prevNode = node[10], nextNode = node[30]
```

```
// node[10]->next = node[30]  
// node[30]->prev = node[10]  
// free(node[20])  
// After: [10] ⇌ [30] ⇌ (circular)
```

### 5.5.2 Delete by Value

```
CDNode* deleteCDLLByValue(CDNode* head, int key) {
    if (!head) return NULL;
    CDNode* current = head;
    do {
        if (current->data == key) return deleteCDLLNode(head,
current);
        current = current->next;
    } while (current != head);
    printf("%d not found\n", key);
    return head;
}
```

## 6. Comparative Summary — All Linked List Types

| Feature                     | SLL               | DLL                            | CSLL         | CDLL                |
|-----------------------------|-------------------|--------------------------------|--------------|---------------------|
| Pointers per node           | 1 (next)          | 2 (prev, next)                 | 1 (next)     | 2 (prev, next)      |
| Last node's next            | NULL              | NULL                           | HEAD         | HEAD                |
| First node's prev           | N/A               | NULL                           | N/A          | TAIL                |
| Traversal direction         | Forward only      | Forward & Backward             | Forward only | Forward & Backward  |
| NULL in list                | Yes (tail's next) | Yes (head's prev, tail's next) | No           | No                  |
| Memory per node             | data + 1 ptr      | data + 2 ptrs                  | data + 1 ptr | data + 2 ptrs       |
| Delete given pointer        | O(n) — find prev  | O(1) — use prev                | O(n)         | O(1)                |
| Insert at end (no tail ptr) | O(n)              | O(n)                           | O(n)         | O(1) via head->prev |

|                             |                      |                                |                         |                           |
|-----------------------------|----------------------|--------------------------------|-------------------------|---------------------------|
| Suitable for                | Simple lists, stacks | Browsers (back/fwd), undo/redo | Round-robin scheduling  | Deque, navigation systems |
| Overflow/infinite loop risk | No (NULL terminates) | No (NULL terminates)           | Yes (must use do-while) | Yes (must use do-while)   |

## 7. Time & Space Complexity Reference

### 7.1 All Operations Across All List Types

| Operation               | SLL    | DLL    | CSLL     | CDLL        |
|-------------------------|--------|--------|----------|-------------|
| Create (n nodes)        | $O(n)$ | $O(n)$ | $O(n)$   | $O(n)$      |
| Traverse                | $O(n)$ | $O(n)$ | $O(n)$   | $O(n)$      |
| Search                  | $O(n)$ | $O(n)$ | $O(n)$   | $O(n)$      |
| Insert at beginning     | $O(1)$ | $O(1)$ | $O(n)^*$ | $O(1)$      |
| Insert at end (no tail) | $O(n)$ | $O(n)$ | $O(n)$   | $O(1)^{**}$ |
| Insert at position i    | $O(i)$ | $O(i)$ | $O(i)$   | $O(i)$      |
| Delete at beginning     | $O(1)$ | $O(1)$ | $O(n)^*$ | $O(1)$      |
| Delete at end           | $O(n)$ | $O(n)$ | $O(n)$   | $O(1)^{**}$ |
| Delete given pointer    | $O(n)$ | $O(1)$ | $O(n)$   | $O(1)$      |
| Delete by value         | $O(n)$ | $O(n)$ | $O(n)$   | $O(n)$      |
| Update at position      | $O(n)$ | $O(n)$ | $O(n)$   | $O(n)$      |

\* CSLL insert/delete at beginning requires traversal to find tail ( $O(n)$ ) unless a tail pointer is maintained. \*\* CDLL insert/delete at end is  $O(1)$  because `head->prev` directly gives the tail.

## 7.2 Space Complexity

| List Type | Space per Node                                | Total Space |
|-----------|-----------------------------------------------|-------------|
| SLL       | <code>sizeof(data) + sizeof(pointer)</code>   | $O(n)$      |
| DLL       | <code>sizeof(data) + 2*sizeof(pointer)</code> | $O(n)$      |
| CSLL      | <code>sizeof(data) + sizeof(pointer)</code>   | $O(n)$      |
| CDLL      | <code>sizeof(data) + 2*sizeof(pointer)</code> | $O(n)$      |

On a 64-bit system: pointer = 8 bytes, int = 4 bytes.

- SLL node:  $4 + 8 = 12$  bytes (+ padding = 16 bytes typically)
- DLL node:  $4 + 8 + 8 = 20$  bytes (+ padding = 24 bytes typically)

## Appendix: Key Pointer Operations Reference

### Insert — Pointer Order (Critical for Exams)

#### SLL Insert After node p:

1. `newNode->next = p->next`
2. `p->next = newNode`

#### DLL Insert After node p:

1. `newNode->next = p->next`

```
2. newNode->prev = p
3. if (p->next != NULL): p->next->prev = newNode
4. p->next = newNode
```

#### CDLL Insert After node p:

```
1. newNode->next = p->next
2. newNode->prev = p
3. p->next->prev = newNode    (no NULL check needed - circular)
4. p->next = newNode
```

#### Delete — Pointer Operations

##### SLL Delete node del (given predecessor prev):

```
1. prev->next = del->next
2. free(del)
```

##### DLL Delete node del:

```
1. if (del->prev): del->prev->next = del->next
2. if (del->next): del->next->prev = del->prev
3. free(del)
```

##### CDLL Delete node del:

```
1. del->prev->next = del->next    (no NULL check - circular)
2. del->next->prev = del->prev    (no NULL check - circular)
```

### 3. `free(del)`

---

Reference: [W3Schools DSA — Linked Lists](#) | [W3Schools DSA — Doubly Linked Lists](#) | [W3Schools DSA — Circular Linked Lists](#)

All algorithms verified against: Cormen et al. CLRS 4th Ed., Knuth TAOCP Vol. 1, GeeksForGeeks DSA curriculum.

---

— **END OF MODULE 2 NOTES** —

Prepared By Francis Reuben R